

Streams

Kategorie	Beschreibung
Definition	Streams sind eine API zur funktionalen Datenverarbeitung großer Datenmengen mit Unterstützung für parallele Verarbeitung. Sie ermöglichen deklarative Pipelines zur Manipulation von Daten.
Paket	<code>java.util.stream</code>
Grundprinzip	Streams bestehen aus einer Quelle, optionalen Zwischenoperationen (intermediate operations) und einer abschließenden Operation (terminal operation).
Quelle (Source)	Streams können aus Collections (<code>List</code> , <code>Set</code>), Arrays, <code>Stream.of(...)</code> , Dateien (<code>Files.lines(...)</code>) oder <code>IntStream.range(...)</code> erzeugt werden.
Beispiel für Erzeugung	<pre>Stream<Integer> stream = List.of(1, 2, 3, 4).stream();</pre>
Interne Iteration	Streams verarbeiten Daten intern mit <code>forEach</code> und optimierten Operationen statt externer Schleifen (<code>for</code> / <code>while</code>).
Lazy Evaluation	Zwischenoperationen werden nicht direkt ausgeführt, sondern erst bei einer Terminal-Operation.
Zwischenoperationen (Intermediate Operations)	Methoden, die einen Stream zurückgeben, um weitere Verarbeitungsschritte zu ermöglichen. Beispiele: <code>filter()</code> , <code>map()</code> , <code>flatMap()</code> , <code>distinct()</code> , <code>sorted()</code> , <code>peek()</code> .
Terminal-Operationen	Methoden, die einen Wert oder eine Datenstruktur zurückgeben und den Stream beenden. Beispiele: <code>collect()</code> , <code>count()</code> , <code>findFirst()</code> , <code>findAny()</code> , <code>forEach()</code> , <code>reduce()</code> , <code>toArray()</code> .
Beispiel mit <code>map()</code>	<pre>List<String> names = List.of("Alice", "Bob").stream().map(String::toUpperCase).toList();</pre>
Beispiel mit <code>filter()</code>	<pre>List<Integer> even = List.of(1,2,3,4).stream().filter(n -> n % 2 == 0).toList();</pre>
Beispiel mit <code>reduce()</code>	<pre>int sum = List.of(1,2,3,4).stream().reduce(0, Integer::sum);</pre>
Kombination von Operationen	Streams können verkettet werden, um komplexe Transformationen zu definieren.
Sammlungsoperationen (collect)	<code>Collectors.toList()</code> , <code>Collectors.toSet()</code> , <code>Collectors.toMap()</code> ermöglichen das Sammeln von Ergebnissen in verschiedene Datenstrukturen.
Parallelverarbeitung (<code>parallelStream</code>)	Streams können parallel verarbeitet werden (<code>parallelStream()</code> oder <code>stream().parallel()</code>), was bei großen Datenmengen Performance-Vorteile bietet.
Unveränderlichkeit	Streams verändern die Originaldaten nicht, sondern erzeugen neue Objekte.

Kategorie	Beschreibung
Endlichkeit	Standardmäßig sind Streams endlich, aber es gibt auch unendliche Streams mit <code>Stream.iterate()</code> und <code>Stream.generate()</code> .
Beispiel für unendlichen Stream	<pre>Stream<Integer> infinite = Stream.iterate(1, n -> n + 1);</pre>
Primitive Streams	<code>IntStream</code> , <code>LongStream</code> , <code>DoubleStream</code> sind spezielle Streams für primitive Datentypen mit Methoden wie <code>sum()</code> , <code>average()</code> .
Boxing & Unboxing	<code>mapToInt()</code> , <code>mapToDouble()</code> helfen, zwischen Objekten und primitiven Typen zu konvertieren.
Limitierung (<code>limit()</code>)	Streams können mit <code>.limit(n)</code> auf eine feste Anzahl von Elementen begrenzt werden.
Kurzschluss-Operationen	Einige Operationen (<code>findFirst()</code> , <code>findAny()</code> , <code>allMatch()</code> , <code>anyMatch()</code> , <code>noneMatch()</code>) können Streams vorzeitig beenden.
Fehlerhandling	Exceptions sollten in Streams mit <code>.map(x -> { try { ... } catch ... })</code> behandelt oder explizit aufgefangen werden.
Performance-Aspekte	Parallelisierung kann Performance verbessern, aber auch Overhead erzeugen (abhängig von Datenmenge und Operationen).
Nachteile	Streams sind nicht wiederverwendbar – nach einer Terminal-Operation ist der Stream geschlossen.